

our results. Some of the images are aligned with the image sides. Picking one of them, you can check the performance of the cropping and classification like this:

```
imname = 'sudokus/sudoku18.jpg'
vername = 'sudokus/sudoku18.sud'
im = array(Image.open(imname).convert('L'))

# find the cell edges
x = find_sudoku_edges(im,axis=0)
y = find_sudoku_edges(im,axis=1)

# crop cells and classify
crops = []
for col in range(9):
    for row in range(9):
        crop = im[y[col]:y[col+1],x[row]:x[row+1]]
        crops.append(compute_feature(crop))

res = svm_predict(loadtxt(vername),map(list,crops),m)[0]
res_im = array(res).reshape(9,9)

print 'Result:'
print res_im
```

The edges are found and then crops are extracted for each cell. The crops are passed to the same feature extraction function used for the training and stored in an array. These feature vectors are classified using `svm_predict()` with the true labels read using `loadtxt()`. The result in your console should be:

```
Accuracy = 100% (81/81) (classification)
Result:
[[ 0.  0.  0.  0.  1.  7.  0.  5.  0.]
 [ 9.  0.  3.  0.  0.  5.  2.  0.  7.]
 [ 0.  0.  0.  0.  0.  0.  4.  0.  0.]
 [ 0.  1.  6.  0.  0.  4.  0.  0.  2.]
 [ 0.  0.  0.  8.  0.  1.  0.  0.  0.]
 [ 8.  0.  0.  5.  0.  0.  6.  4.  0.]
 [ 0.  0.  9.  0.  0.  0.  0.  0.  0.]
 [ 7.  0.  2.  1.  0.  0.  8.  0.  9.]
 [ 0.  5.  0.  2.  3.  0.  0.  0.  0.]
```

Now, this was one of the easier images. Try some of the others and see what the errors look like and where the classifier makes mistakes.

If you plot the crops using a 9×9 subplot, they should look like the right image of Figure 8-7.

Rectifying Images

If you are happy with the performance of your classifier, the next challenge is to apply it to non-aligned images. We will end our Sudoku example with a simple way of rectifying an image given that the four outer corner points of the grid have been detected or marked manually. The left image in Figure 8-8 shows an example of a Sudoku image with strong perspective effects.

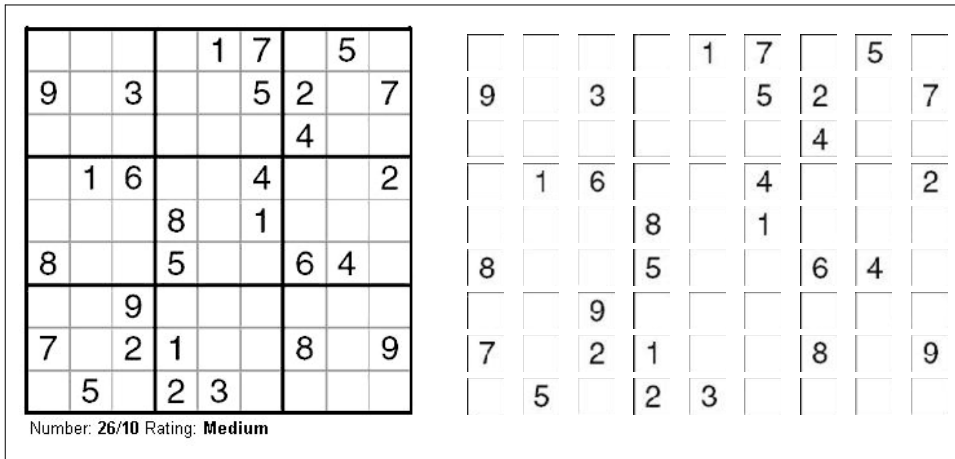


Figure 8-7. An example of detecting and cropping the fields of a Sudoku grid: image of a Sudoku grid (left); the 9×9 cropped images of the individual cells to be sent to the OCR classifier (right).

A homography can map the grid to align the edges as in the examples above, all we need to do is estimate the transform. The example below shows the case of manually marking the four corner points and then warping the image to a square target image of 1000×1000 pixels:

```
from scipy import ndimage
import homography

imname = 'sudoku8.jpg'
im = array(Image.open(imname).convert('L'))

# mark corners
figure()
imshow(im)
gray()
x = ginput(4)

# top left, top right, bottom right, bottom left
fp = array([array([p[1],p[0],1]) for p in x]).T
tp = array([[0,0,1],[0,1000,1],[1000,1000,1],[1000,0,1]]).T

# estimate the homography
H = homography.H_from_points(tp,fp)

# helper function for geometric_transform
def warpfcn(x):
    x = array([x[0],x[1],1])
    xt = dot(H,x)
    xt = xt/xt[2]
    return xt[0],xt[1]

# warp image with full perspective transform
im_g = ndimage.geometric_transform(im,warpfcn,(1000,1000))
```

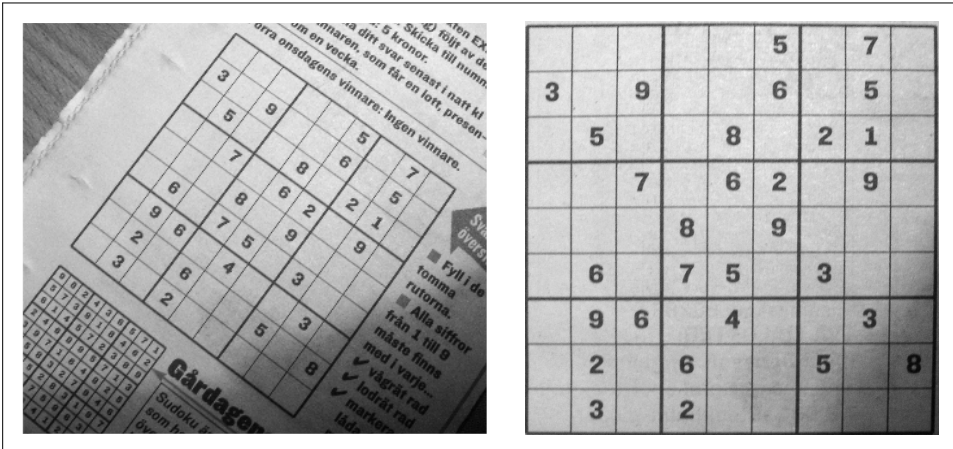


Figure 8-8. An example of rectifying an image using a full perspective transform: the original image with the four corners of the Sudoku marked (left); rectified image warped to a square image of 1000×1000 pixels (right).

In most of these sample images an affine transform, as we used in Chapter 3, is not enough. Here we instead used the more general transform function `geometric_transform()` from `scipy.ndimage`. This function takes a 2D to 2D mapping instead of a transform matrix, so we need to use a helper function (using a piecewise affine warp on triangles will introduce artifacts in this case). The warped image is shown to the right in Figure 8-8.

This concludes our Sudoku OCR example. There are many improvements to be made and alternatives to investigate. Some are mentioned in the following exercises; the rest we leave to you.

Exercises

1. The performance of the kNN classifier depends on the value of k . Try to vary this number and see how the accuracy changes. Plot the decision boundaries of the 2D point sets to see how they change.
2. The hand gesture data set in Figure 8-3 also contains images with more complex background (in the “complex/” folders). Try to train and test a classifier on these images. What is the difference in performance? Can you suggest improvements to the image descriptor?
3. Try to vary the number of dimensions after PCA projection of the gesture recognition features for the Bayes classifier. What is a good choice? Plot the singular values S , they should give a typical “knee” shaped curve like the one shown in Figure 8-9. A good compromise between ability to generate the variability of the training data and keeping the number of dimensions low is usually found at a number before the curve flattens out.

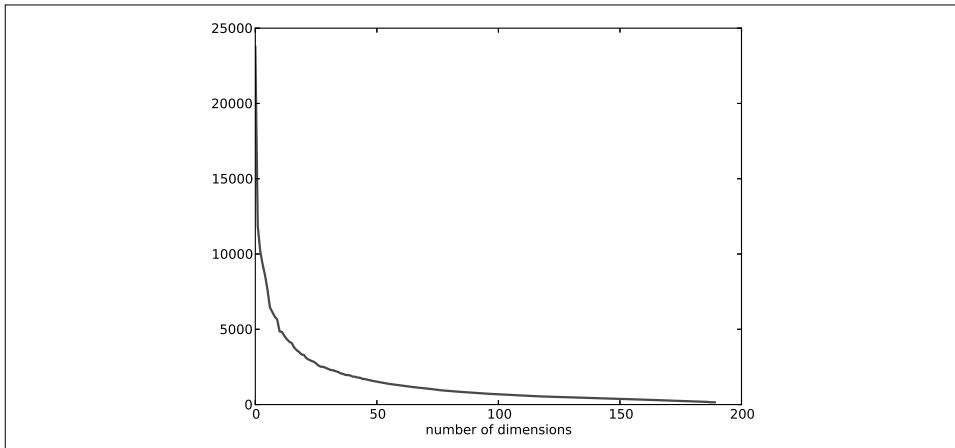


Figure 8-9. Graph for Exercise 3.

4. Modify the Bayes classifier to use a different probability model than Gaussian distributions. For example, try using the frequency counts of each feature in the training set. Compare the results to using a Gaussian distribution for some different datasets.
5. Experiment with using non-linear SVMs for the gesture recognition problem. Try polynomial kernels and increase the degree (using the “-d” parameter) incrementally. What happens to the classification performance on the training set and the test set? With a non-linear classifier, there is a risk of training and optimizing it for a specific set so that performance is close to perfect on the training set, but the classifier has poor performance on other test sets. This phenomenon of breaking the generalization capabilities of a classifier is called *overfitting* and should be avoided.
6. Try some more advanced feature vectors for the Sudoku character recognition problem. If you need inspiration, look at [4].
7. Implement a method for automatically aligning the Sudoku grid. Try, for example, feature detection with RANSAC, line detection, or detecting the cells using morphological and measurement operations from `scipy.ndimage` (<http://docs.scipy.org/doc/scipy/reference/ndimage.html>). Bonus task: Solve the rotation ambiguity of finding the “up” direction. For example, you could try rotating the rectified grid and let the OCR classifier’s accuracy vote for the best orientation.
8. For a more challenging classification problem than the Sudoku digits, take a look at the MNIST database of *handwritten digits* <http://yann.lecun.com/exdb/mnist/>. Try to extract some features and apply SVM to that set. Check where your performance ends up on the ranking of best methods (some are insanely good).
9. If you want to dive deeper into classifiers and machine learning algorithms, take a look at the `scikit.learn` package (<http://scikit-learn.org/>) and try some of the algorithms on the data in this chapter.

Image Segmentation

Image segmentation is the process of partitioning an image into meaningful regions. Regions can be foreground versus background or individual objects in the image. The regions are constructed using some feature such as color, edges, or neighbor similarity. In this chapter we will look at some different techniques for segmentation.

9.1 Graph Cuts

A *graph* is a set of nodes (sometimes called vertices) with edges between them. See Figure 9-1 for an example.¹ The edges can be directed (as illustrated with arrows in Figure 9-1) or undirected, and may have weights associated with them.

A *graph cut* is the partitioning of a directed graph into two disjoint sets. Graph cuts can be used for solving many different computer vision problems like stereo depth reconstruction, image stitching, and image segmentation. By creating a graph from image pixels and their neighbors and introducing an energy or a “cost,” it is possible to use a graph cut process to segment an image in two or more regions. The basic idea is that similar pixels that are also close to each other should belong to the same partition.

The cost of a graph cut C (where C is a set of edges) is defined as the sum of the edge weights of the cuts

$$E_{cut} = \sum_{(i,j) \in C} w_{ij}, \quad (9.1)$$

where w_{ij} is the weight of the edge (i, j) from node i to node j in the graph and the sum is taken over all edges in the cut C .

The idea behind graph cut segmentation is to partition a graph representation of the image such that the cut cost E_{cut} is minimized. In this graph representation, two additional nodes, a source and a sink node, are added to the graph and only cuts that separate the source and sink are considered.

¹ You also saw graphs in action in Section 2.3. This time we are going to use them to partition images.

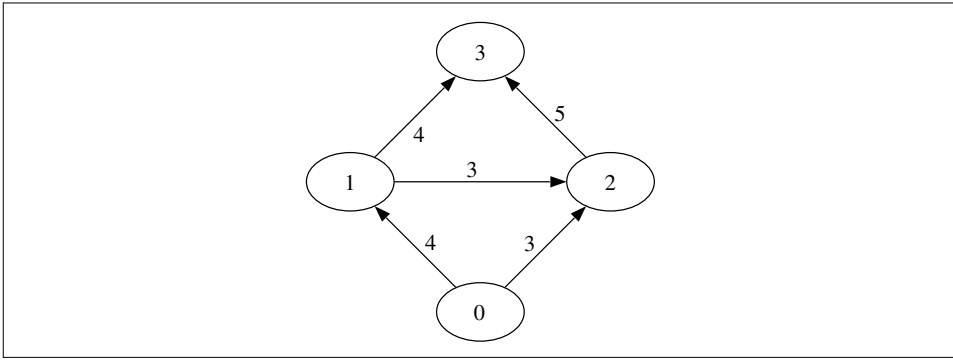


Figure 9-1. A simple directed graph created using `python-graph`.

Finding the *minimum cut* (or *min cut*) is equivalent to finding the *maximum flow* (or *max flow*) between the source and the sink (see [2] for details). There are efficient algorithms for solving these max flow/min cut problems.

For our graph cut examples we will use the `python-graph` package. This package contains many useful graph algorithms. The website with downloads and documentation is <http://code.google.com/p/python-graph/>. We will need the function `maximum_flow()`, which computes the max flow/min cut using the Edmonds-Karp algorithm (http://en.wikipedia.org/wiki/Edmonds-Karp_algorithm). The good thing about using a package written fully in Python is ease of installation and compatibility; the downside is speed. Performance is adequate for our purposes, but for anything but small images, a faster implementation is needed.

Here's a simple example of using `python-graph` to compute the max flow/min cut of a small graph.²

```

from pygraph.classes.digraph import digraph
from pygraph.algorithms.minmax import maximum_flow

gr = digraph()
gr.add_nodes([0,1,2,3])

gr.add_edge((0,1), wt=4)
gr.add_edge((1,2), wt=3)
gr.add_edge((2,3), wt=5)
gr.add_edge((0,2), wt=3)
gr.add_edge((1,3), wt=4)

flows,cuts = maximum_flow(gr,0,3)
print 'flow is:', flows
print 'cut is:', cuts

```

First, a directed graph is created with four nodes with index 0 . . . 3. Then the edges are added using `add_edge()` with an edge weight specified. This will be used as the

² Same graph as the example at http://en.wikipedia.org/wiki/Max-flow_min-cut_theorem.

maximum flow capacity of the edge. The maximum flow is computed with node 0 as source and node 3 as sink. The flow and the cuts are printed and should look like this:

```
flow is: {(0, 1): 4, (1, 2): 0, (1, 3): 4, (2, 3): 3, (0, 2): 3}
cut is: {0: 0, 1: 1, 2: 1, 3: 1}
```

These two python dictionaries contain the flow through each edge and the label for each node: 0 for the part of the graph containing the source, 1 for the nodes connected to the sink. You can verify manually that the cut is indeed the minimum. The graph is shown in Figure 9-1.

Graphs from Images

Given a neighborhood structure, we can define a graph using the image pixels as nodes. Here we will focus on the simplest case of 4-neighborhood of pixels and two image regions (which we can call foreground and background). A *4-neighborhood* is where a pixel is connected to the pixels directly above, below, left, and right.³

In addition to the pixel nodes, we will also need two special nodes: a “source” node and a “sink” node, representing the foreground and background, respectively. We will use a simple model where all pixels are connected to the source and the sink.

Here’s how to build the graph:

- Every pixel node has an incoming edge from the source node.
- Every pixel node has an outgoing edge to the sink node.
- Every pixel node has one incoming and one outgoing edge to each of its neighbors.

To determine the weights on these edges, you need a segmentation model that determines the edge weights (representing the maximum flow allowed for that edge) between pixels and between pixels and the source and sink. As before, we call the edge weight between pixel i and pixel j , w_{ij} . Let’s call the weight from the source to pixel i , w_{si} , and from pixel i to the sink, w_{it} .

Let’s look at using a naive Bayesian classifier from Section 8.2 on the color values of the pixels. Given that we have trained a Bayes classifier on foreground and background pixels (from the same image or from other images), we can compute the probabilities $p_F(I_i)$ and $p_B(I_i)$ for the foreground and background. Here I_i is the color vector of pixel i .

We can now create a model for the edge weights as follows:

$$w_{si} = \frac{p_F(I_i)}{p_F(I_i) + p_B(I_i)}$$

$$w_{it} = \frac{p_B(I_i)}{p_F(I_i) + p_B(I_i)}$$

$$w_{ij} = \kappa e^{-|I_i - I_j|^2 / \sigma}.$$

³ Another common option is 8-neighborhood, where the diagonal pixels are also connected.

With this model, each pixel is connected to the foreground and background (source and sink) with weights equal to a normalized probability of belonging to that class. The w_{ij} describe the pixel similarity between neighbors; similar pixels have weight close to κ , dissimilar close to 0. The parameter σ determines how fast the values decay toward zero with increasing dissimilarity.

Create a file *graphcut.py* and add the following function that creates this graph from an image:

```
from pygraph.classes.digraph import digraph
from pygraph.algorithms.minmax import maximum_flow

import bayes

def build_bayes_graph(im, labels, sigma=1e2, kappa=2):
    """ Build a graph from 4-neighborhood of pixels.
        Foreground and background is determined from
        labels (1 for foreground, -1 for background, 0 otherwise)
        and is modeled with naive Bayes classifiers."""

    m, n = im.shape[:2]

    # RGB vector version (one pixel per row)
    vim = im.reshape((-1, 3))

    # RGB for foreground and background
    foreground = im[labels==1].reshape((-1, 3))
    background = im[labels==-1].reshape((-1, 3))
    train_data = [foreground, background]

    # train naive Bayes classifier
    bc = bayes.BayesClassifier()
    bc.train(train_data)

    # get probabilities for all pixels
    bc_labels, prob = bc.classify(vim)
    prob_fg = prob[0]
    prob_bg = prob[1]

    # create graph with m*n+2 nodes
    gr = digraph()
    gr.add_nodes(range(m*n+2))

    source = m*n # second to last is source
    sink = m*n+1 # last node is sink

    # normalize
    for i in range(vim.shape[0]):
        vim[i] = vim[i] / linalg.norm(vim[i])

    # go through all nodes and add edges
    for i in range(m*n):
        # add edge from source
        gr.add_edge((source, i), wt=(prob_fg[i]/(prob_fg[i]+prob_bg[i])))
```

```

# add edge to sink
gr.add_edge((i,sink), wt=(prob_bg[i]/(prob_fg[i]+prob_bg[i])))

# add edges to neighbors
if i%n != 0: # left exists
    edge_wt = kappa*exp(-1.0*sum((vim[i]-vim[i-1])**2)/sigma)
    gr.add_edge((i,i-1), wt=edge_wt)
if (i+1)%n != 0: # right exists
    edge_wt = kappa*exp(-1.0*sum((vim[i]-vim[i+1])**2)/sigma)
    gr.add_edge((i,i+1), wt=edge_wt)
if i//n != 0: # up exists
    edge_wt = kappa*exp(-1.0*sum((vim[i]-vim[i-n])**2)/sigma)
    gr.add_edge((i,i-n), wt=edge_wt)
if i//n != m-1: # down exists
    edge_wt = kappa*exp(-1.0*sum((vim[i]-vim[i+n])**2)/sigma)
    gr.add_edge((i,i+n), wt=edge_wt)

return gr

```

Here we used a label image with values 1 for foreground training data and -1 for background training data. Based on this labeling, a Bayes classifier is trained on the RGB values. Then classification probabilities are computed for each pixel. These are then used as edge weights for the edges going from the source and to the sink. A graph with $n * m + 2$ nodes is created. Note the index of the source and sink; we choose them as the last two to simplify the indexing of the pixels.

To visualize the labeling overlaid on the image we can use the function `contourf()`, which fills the regions between contour levels of an image (in this case the label image). The *alpha* variable sets the transparency. Add the following function to *graphcut.py*:

```

def show_labeling(im,labels):
    """ Show image with foreground and background areas.
        labels = 1 for foreground, -1 for background, 0 otherwise. """

    imshow(im)
    contour(labels,[-0.5,0.5])
    contourf(labels,[-1,-0.5],colors='b',alpha=0.25)
    contourf(labels,[0.5,1],colors='r',alpha=0.25)
    axis('off')

```

Once the graph is built, it needs to be cut at the optimal location. The following function computes the min cut and reformats the output to a binary image of pixel labels:

```

def cut_graph(gr,imsize):
    """ Solve max flow of graph gr and return binary
        labels of the resulting segmentation. """

    m,n = imsize
    source = m*n # second to last is source
    sink = m*n+1 # last is sink

    # cut the graph
    flows,cuts = maximum_flow(gr,source,sink)

```

```

# convert graph to image with labels
res = zeros(m*n)
for pos,label in cuts.items()[:-2]: #don't add source/sink
    res[pos] = label

return res.reshape((m,n))

```

Again, note the indices for the source and sink. We need to take the size of the image as input to compute these indices and to reshape the output before returning the segmentation. The cut is returned as a dictionary, which needs to be copied to an image of segmentation labels. This is done using the *.items()* method that returns a list of (key, value) pairs. Again we skip the last two elements of that list.

Let's see how to use these functions for segmenting an image. The following is a complete example of reading an image and creating a graph with class probabilities estimated from two rectangular image regions:

```

from scipy.misc import imread
import graphcut

im = array(Image.open('empire.jpg'))
im = imresize(im,0.07,interp='bilinear')
size = im.shape[:2]

# add two rectangular training regions
labels = zeros(size)
labels[3:18,3:18] = -1
labels[-18:-3,-18:-3] = 1

# create graph
g = graphcut.build_bayes_graph(im,labels,kappa=1)

# cut the graph
res = graphcut.cut_graph(g,size)

figure()
graphcut.show_labeling(im,labels)

figure()
imshow(res)
gray()
axis('off')

show()

```

We use the *imresize()* function to make the image small enough for our Python graph library, in this case uniform scaling to 7% of the original size. The graph is cut and the result plotted together with an image showing the training regions. Figure 9-2 shows the training regions overlaid on the image and the final segmentation result.

The variable *kappa* (κ in the equations) determines the relative weight of the edges between neighboring pixels. The effect of changing *kappa* can be seen in Figure 9-3. With increasing value, the segmentation boundary will be smoother and details will be lost. Choosing the right value is up to you. The right value will depend on your application and the type of result you desire.

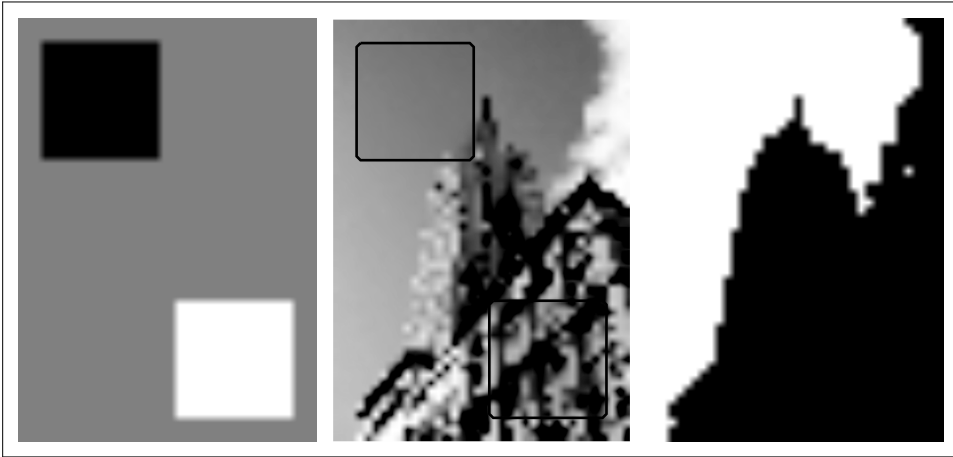


Figure 9-2. An example of graph cut segmentation using a Bayesian probability model. Image is downsampled to size 54×38 . Label image for model training (left); training regions shown on the image (middle); segmentation (right).

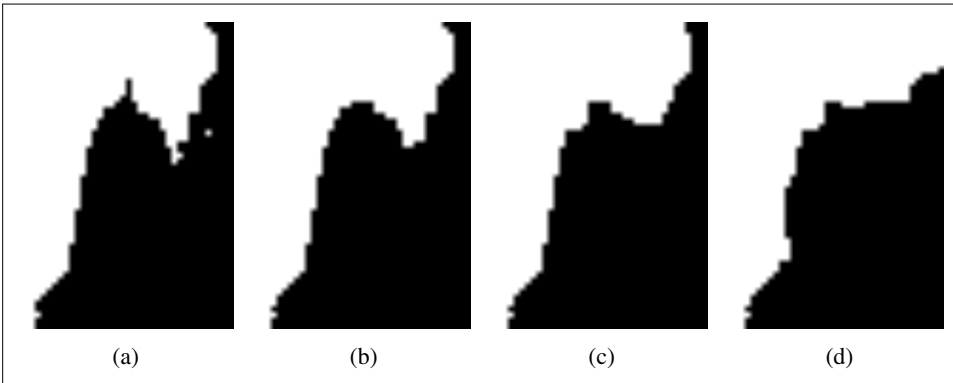


Figure 9-3. The effect of changing the relative weighting between pixel similarity and class probability. The same segmentation as in Figure 9-2 with: (a) $\kappa = 1$, (b) $\kappa = 2$, (c) $\kappa = 5$, and (d) $\kappa = 10$.

Segmentation with User Input

Graph cut segmentation can be combined with user input in a number of ways. For example, a user can supply markers for foreground and background by drawing on an image. Another way is to select a region that contains the foreground with a bounding box or using a “lasso” tool.

Let’s look at this last example using some images from the Grab Cut dataset from Microsoft Research Cambridge; see [27] and Appendix B.5 for details.

These images come with ground truth labels for measuring segmentation performance. They also come with annotations simulating a user selecting a rectangular image region or drawing on the image with a “lasso” type tool to mark foreground and background.

We can use these user inputs to get training data and apply graph cuts to segment the image guided by the user input.

The user input is encoded in bitmap images with the following meaning:

Pixel value	Meaning
0, 64	background
128	unknown
255	foreground

Here's a complete code example of loading an image and annotations and passing that to our graph cut segmentation routine:

```
from scipy.misc import imresize
import graphcut

def create_msr_labels(m,lasso=False):
    """ Create label matrix for training from
        user annotations. """

    labels = zeros(im.shape[:2])

    # background
    labels[m==0] = -1
    labels[m==64] = -1

    # foreground
    if lasso:
        labels[m==255] = 1
    else:
        labels[m==128] = 1

    return labels

# load image and annotation map
im = array(Image.open('376043.jpg'))
m = array(Image.open('376043.bmp'))

# resize
scale = 0.1
im = imresize(im,scale,interp='bilinear')
m = imresize(m,scale,interp='nearest')

# create training labels
labels = create_msr_labels(m,False)

# build graph using annotations
g = graphcut.build_bayes_graph(im,labels,kappa=2)

# cut graph
res = graphcut.cut_graph(g,im.shape[:2])

# remove parts in background
res[m==0] = 1
res[m==64] = 1
```

```
# plot the result
figure()
imshow(res)
gray()
xticks([])
yticks([])
savefig('labelplot.pdf')
```

First, we define a helper function to read the annotation images and format them so we can pass them to our function for training background and foreground models. The bounding rectangles contain only background labels. In this case, we set the foreground training region to the whole “unknown” region (the inside of the rectangle). Next, we build the graph and cut it. Since we have user input, we remove results that have any foreground in the marked background area. Last, we plot the resulting segmentation and remove the tick markers by setting them to an empty list. That way, we get a nice bounding box (otherwise the boundaries of the image will be hard to see in this black-and-white plot).

Figure 9-4 shows some results using RGB vector as feature with the original image, a downsampled mask, and downsampled resulting segmentation. The image on the right is the plot generated by the script above.

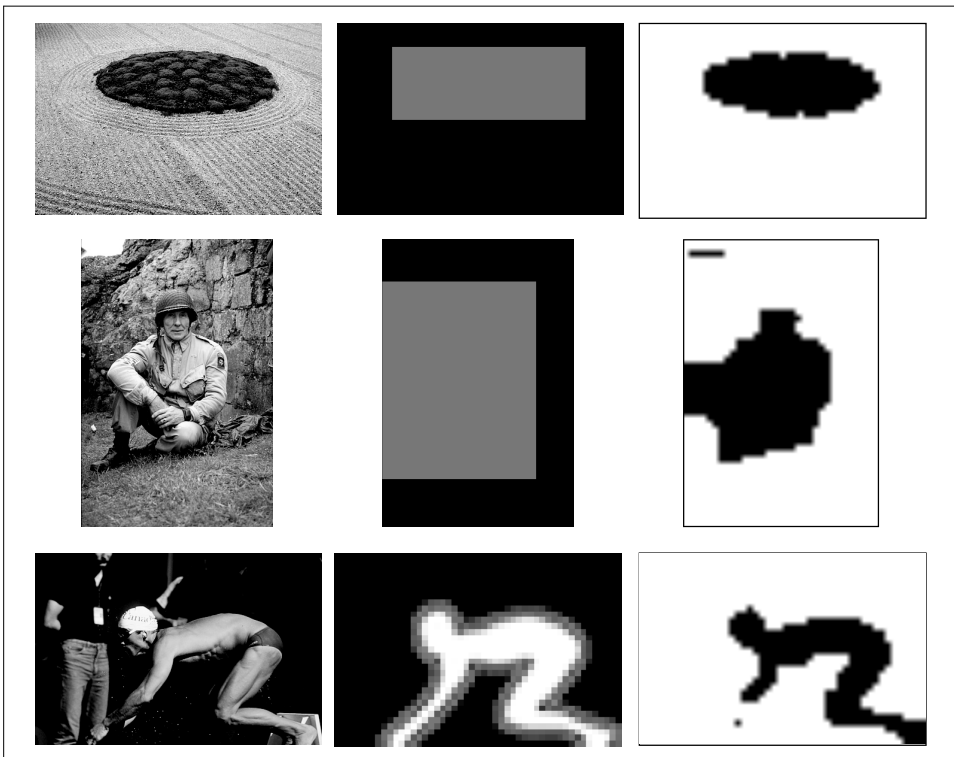


Figure 9-4. Sample graph cut segmentation results using images from the Grab Cut data set: original image, downsampled (left); mask used for training (middle); resulting segmentation using RGB values as feature vectors (right).

9.2 Segmentation Using Clustering

The graph cut formulation in the previous section solves the segmentation problem by finding a discrete solution using max flow/min cut over an image graph. In this section we will look at an alternative way to cut the image graph. The *normalized cut* algorithm, based on spectral graph theory, combines pixel similarities with spatial proximity to segment the image.

The idea comes from defining a cut cost that takes into account the size of the groups and “normalizes” the cost with the size of the partitions. The normalized cut formulation modifies the cut cost of equation (9.1) to

$$E_{ncut} = \frac{E_{cut}}{\sum_{i \in A} w_{ix}} + \frac{E_{cut}}{\sum_{j \in B} w_{jx}},$$

where A and B indicate the two sets of the cut and the sums add the weights from A and B , respectively, to all other nodes in the graph (which are pixels in the image in this case). This sum is called the *association* and for images where pixels have the same number of connections to other pixels, it is a rough measure of the size of the partitions. In the paper [32], the cost function above was introduced together with an algorithm for finding a minimizer. The algorithm is derived for two-class segmentation and will be described next.

Define W as the edge weight matrix with elements w_{ij} containing the weight of the edge connecting pixel i with pixel j . Let D be the diagonal matrix of the row sums of S , $D = \text{diag}(d_i)$, $d_i = \sum_j w_{ij}$ (same as in Section 6.3). The normalized cut segmentation is obtained as the minimum of the optimization problem

$$\min_{\mathbf{y}} \frac{\mathbf{y}^T (D - W) \mathbf{y}}{\mathbf{y}^T D \mathbf{y}},$$

where the vector $\mathbf{y}$ contains the discrete labels that satisfy the constraints $y_i \in \{1, -b\}$ for some constant b (meaning that $\mathbf{y}$ only takes two discrete values) and $\mathbf{y}^T D \mathbf{y}$ sum to zero. Because of these constraints, this is not easily solvable.⁴

However, by relaxing the constraints and letting $\mathbf{y}$ take any real value, the problem becomes an eigenvalue problem that is easily solved. The drawback is that you need to threshold or cluster the output to make it a discrete segmentation again.

Relaxing the problem results in solving for eigenvectors of a Laplacian matrix

$$L = D^{-1/2} W D^{-1/2},$$

just like the spectral clustering case. The only remaining difficulty is now to define the between-pixel edge weights w_{ij} . Normalized cuts have many similarities to spectral clustering and the underlying theory overlaps somewhat. See [32] for an explanation and the details.

⁴ In fact, this problem is NP-hard.

Let's use the edge weights from the original normalized cuts paper [32]. The edge weight connecting two pixels i and j is given by

$$w_{ij} = e^{-|I_i - I_j|^2 / \sigma_g} e^{-|\mathbf{x}_i - \mathbf{x}_j|^2 / \sigma_d}.$$

The first part measures the pixel similarity between the pixels with I_i and I_j denoting either the RGB vectors or the grayscale values. The second part measures the proximity between the pixels in the image with $\mathbf{x}_i$ and $\mathbf{x}_j$ denoting the coordinate vector of each pixel. The scaling factors σ_g and σ_d determine the relative scales and how fast each component approaches zero.

Let's see what this looks like in code. Add the following function to a file `ncut.py`:

```
def ncut_graph_matrix(im, sigma_d=1e2, sigma_g=1e-2):
    """ Create matrix for normalized cut. The parameters are
        the weights for pixel distance and pixel similarity. """

    m, n = im.shape[:2]
    N = m*n

    # normalize and create feature vector of RGB or grayscale
    if len(im.shape)==3:
        for i in range(3):
            im[:, :, i] = im[:, :, i] / im[:, :, i].max()
        vim = im.reshape((-1, 3))
    else:
        im = im / im.max()
        vim = im.flatten()

    # x,y coordinates for distance computation
    xx, yy = meshgrid(range(n), range(m))
    x, y = xx.flatten(), yy.flatten()

    # create matrix with edge weights
    W = zeros((N, N), 'f')
    for i in range(N):
        for j in range(i, N):
            d = (x[i]-x[j])**2 + (y[i]-y[j])**2
            W[i, j] = W[j, i] = exp(-1.0*sum((vim[i]-vim[j])**2)/sigma_g) * exp(-d/sigma_d)

    return W
```

This function takes an image array and creates a feature vector using either RGB values or grayscale values depending on the input image. Since the edge weights contain a distance component, we use `meshgrid()` to get the x and y values for each pixel feature vector. Then the function loops over all N pixels and fills out the values in the $N \times N$ normalized cut matrix W .

We can compute the segmentation either by sequentially cutting each eigenvector or by taking a number of eigenvectors and apply clustering. We chose the second approach, which also works without modification for any number of segments. We take the top $ndim$ eigenvectors of the Laplacian matrix corresponding to W and cluster the pixels.

The following function implements the clustering. As you can see, it is almost the same as the spectral clustering example in Section 6.3:

```
from scipy.cluster.vq import *

def cluster(S,k,ndim):
    """ Spectral clustering from a similarity matrix."""

    # check for symmetry
    if sum(abs(S-S.T)) > 1e-10:
        print 'not symmetric'

    # create Laplacian matrix
    rowsum = sum(abs(S),axis=0)
    D = diag(1 / sqrt(rowsum + 1e-6))
    L = dot(D,dot(S,D))

    # compute eigenvectors of L
    U,sigma,V = linalg.svd(L)

    # create feature vector from ndim first eigenvectors
    # by stacking eigenvectors as columns
    features = array(V[:ndim]).T

    # k-means
    features = whiten(features)
    centroids,distortion = kmeans(features,k)
    code,distance = vq(features,centroids)

    return code,V
```

Here we used the *k*-means clustering algorithm (see Section 6.1 for details) to group the pixels based on the values in the eigenvector images. You could try any clustering algorithm or grouping criteria if you feel like experimenting with the results.

Now we are ready to try this on some sample images. The following script shows a complete example:

```
import ncut
from scipy.misc import imread

im = array(Image.open('C-uniform03.ppm'))
m,n = im.shape[:2]

# resize image to (wid,wid)
wid = 50
rim = imresize(im,(wid,wid),interp='bilinear')
rim = array(rim,'f')

# create normalized cut matrix
A = ncut.ncut_graph_matrix(rim,sigma_d=1,sigma_g=1e-2)

# cluster
code,V = ncut.cluster(A,k=3,ndim=3)
```

```
# reshape to original image size
codeim = imresize(code.reshape(wid,wid),(m,n),interp='nearest')

# plot result
figure()
imshow(codeim)
gray()
show()
```

Here we resize the image to a fixed size (50×50 in this example) in order to make the eigenvector computation fast enough. The NumPy `linalg.svd()` function is not fast enough to handle large matrices (and sometimes gives inaccurate results for too large matrices). We use bilinear interpolation when resizing the image, but nearest neighbor interpolation when resizing the resulting segmentation label image, since we don't want to interpolate the class labels. Note the use of first reshaping the one-dimensional array to (wid,wid) followed by resizing to the original image size.

In the example, we used one of the hand gesture images from the Static Hand Posture Database (see Section 8.1 for more details) with $k = 3$. The resulting segmentation is shown in Figure 9-5 together with the first four eigenvectors.

The eigenvectors are returned as the array V in the example and can be visualized as images like this:

```
imshow(imresize(V[i].reshape(wid,wid),(m,n),interp='bilinear'))
```

This will show eigenvector i as an image at the original image size.

Figure 9-6 shows some more examples using the same script above. The airplane image is from the “airplane” category in the Caltech 101 dataset. For these examples, we kept the parameters σ_d and σ_g to the same values as above. Changing them can give you

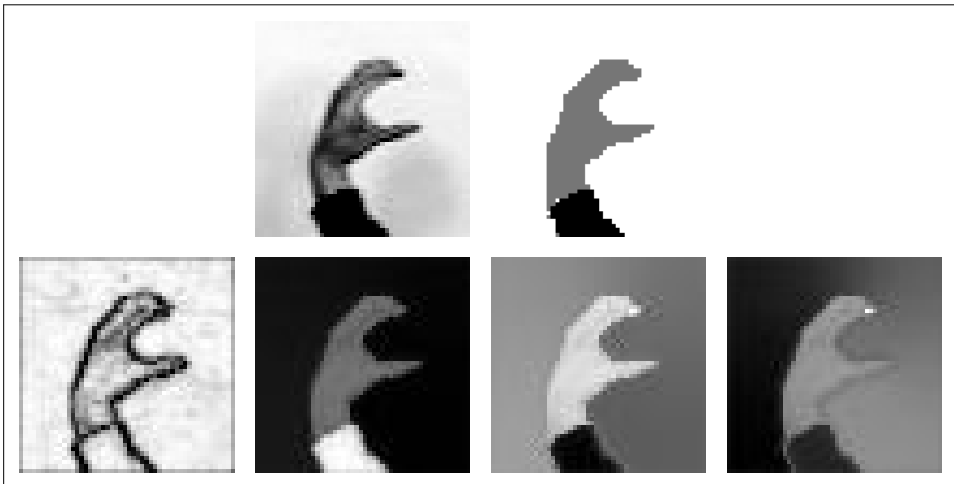


Figure 9-5. Image segmentation using the normalized cuts algorithm: the original image and the resulting three-class segmentation (top); the first four eigenvectors of the graph similarity matrix (bottom).

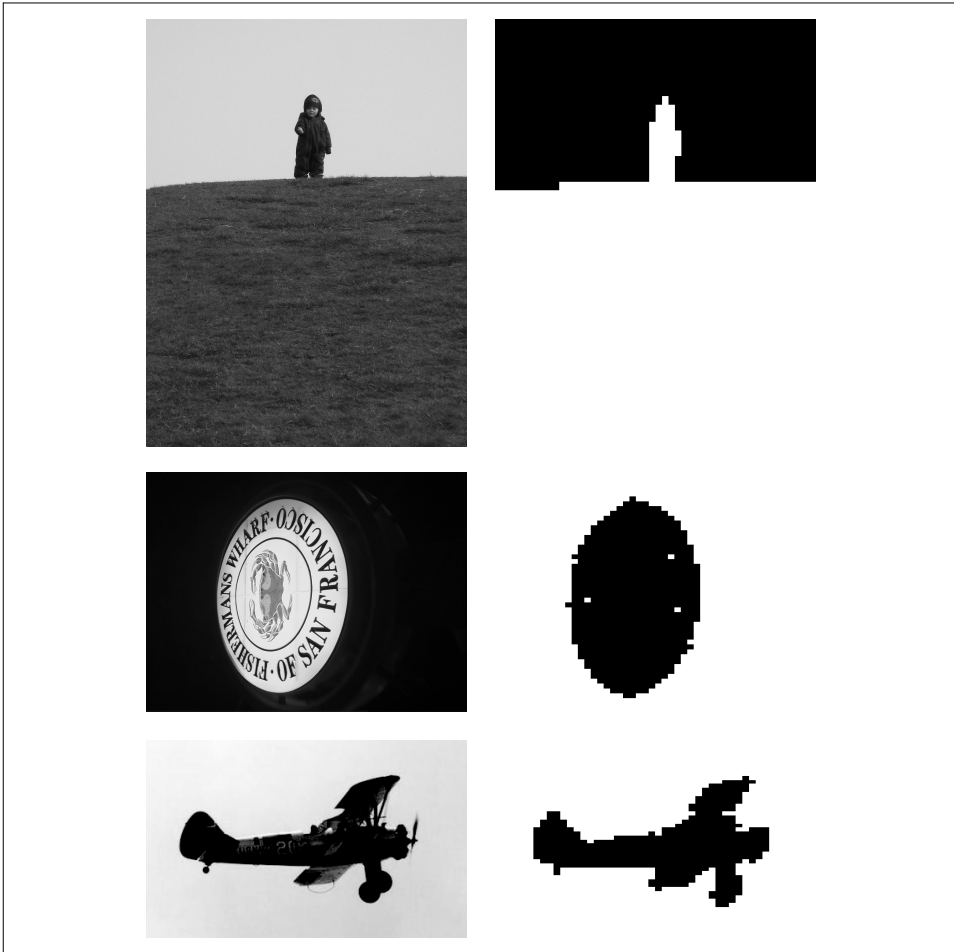


Figure 9-6. Examples of two-class image segmentation using the normalized cuts algorithm: original image (left); segmentation result (right).

smoother, more regularized results and quite different eigenvector images. We leave the experimentation to you.

It is worth noting that even for these fairly simple examples, a thresholding of the image would not have given the same result; neither would clustering the RGB or graylevel values. This is because neither of these take the pixel neighborhoods into account.

9.3 Variational Methods

In this book, you have seen a number of examples of minimizing a cost or energy to solve computer vision problems. In the previous sections it was minimizing the cut in a

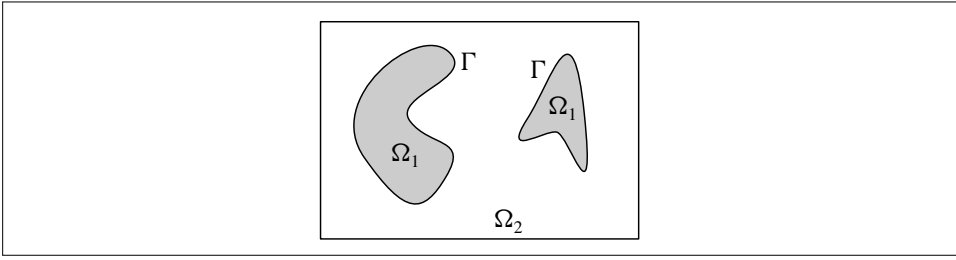


Figure 9-7. The piecewise constant Chan-Vese segmentation model.

graph, but we also saw examples like the ROF de-noising, k -means, and support vector machines. These are examples of optimization problems.

When the optimization is taken over functions, the problems are called *variational problems*, and algorithms for solving such problems are called *variational methods*. Let's look at a simple and effective variational model.

The *Chan-Vese segmentation* model [6] assumes a piecewise constant image model for the image regions to be segmented. Here we will focus on the case of two regions, for example foreground and background, but the model extends to multiple regions as well; see, for example, [38]. The model can be described as follows.

If we let a collection of curves Γ separate the image into two regions Ω_1 and Ω_2 as in Figure 9-7, the segmentation is given by minima of the Chan-Vese model energy

$$E(\Gamma) = \lambda \text{length}(\Gamma) + \int_{\Omega_1} (I - c_1)^2 d\mathbf{x} + \int_{\Omega_2} (I - c_2)^2 d\mathbf{x},$$

which measures the deviation from the constant graylevels in each region, c_1 and c_2 . Here the integrals are taken over each region and the length of the separating curves are there to prefer smoother solutions.

With a piecewise constant image $U = \chi_1 c_1 + \chi_2 c_2$, this can be re-written as

$$E(\Gamma) = \lambda \frac{|c_1 - c_2|}{2} \int |\nabla U| d\mathbf{x} + \|I - U\|^2,$$

where χ_1 and χ_2 are characteristic (indicator) functions for the two regions.⁵ This transformation is non-trivial and requires some heavy mathematics that are not needed for understanding and that are well outside the scope of this book.

The point is that this equation is now the same as the ROF equation (1.1) with λ replaced by $\lambda|c_1 - c_2|$. The only difference is that in the Chan-Vese case we are looking for an image U that is piecewise constant. It can be shown that thresholding the ROF solution will give a good minimizer. The interested reader can check [8] for the details.

⁵ Characteristic functions are 1 in the region and 0 outside.

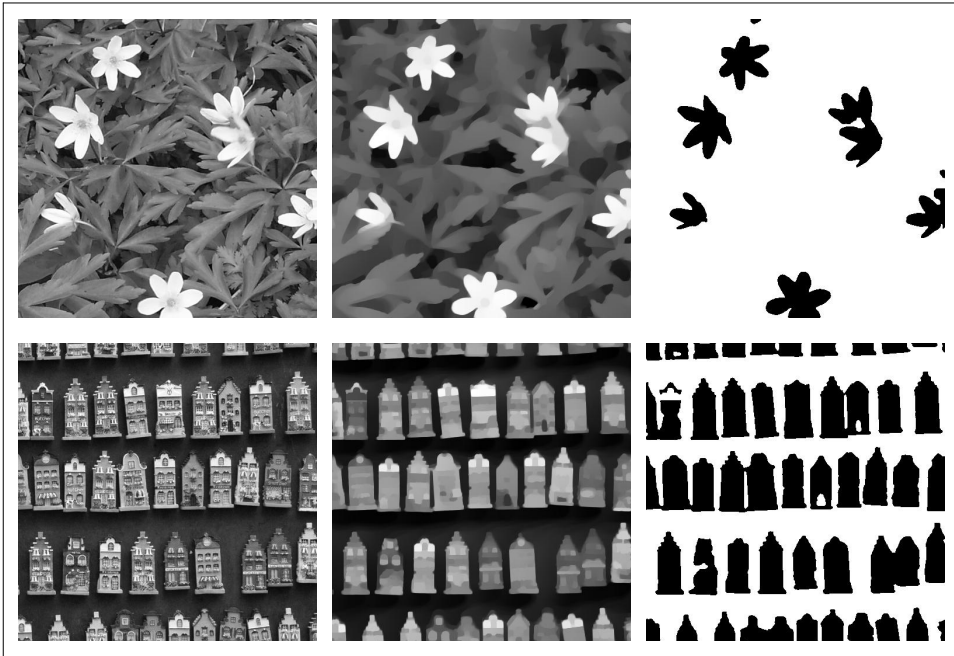


Figure 9-8. Examples image segmentation by minimizing the Chan-Vese model using ROF de-noising: (a) original image; (b) image after ROF de-noising; (c) final segmentation.

Minimizing the Chan-Vese model now becomes a ROF de-noising followed by thresholding:

```
import rof

im = array(Image.open('ceramic-houses_t0.png').convert("L"))
U,T = rof.denoise(im,im,tolerance=0.001)
t = 0.4 #threshold

import scipy.misc
scipy.misc.imsave('result.pdf',U < t*U.max())
```

In this case, we turn down the tolerance threshold for stopping the ROF iterations to make sure we get enough iterations. Figure 9-8 shows the result on two rather difficult images.

Exercises

1. It is possible to speed up computation for the graph cut optimization by reducing the number of edges. This graph construction is described in Section 4.2 of [16]. Try this out and measure the difference in graph size and in segmentation time compared to the simpler construction we used.

2. Create a user interface or simulate a user selecting regions for graph cut segmentation. Then try “hard coding” background and foreground by setting weights to some large value.
3. Change the feature vector in the graph cut segmentation from a RGB vector to some other descriptor. Can you improve on the segmentation results?
4. Implement an iterative segmentation approach using graph cut where a current segmentation is used to train new foreground and background models for the next. Does it improve segmentation quality?
5. The Microsoft Research Grab Cut dataset contains ground truth segmentation maps. Implement a function that measures the segmentation error and evaluate different settings and some of the ideas in the exercises above.
6. Try to vary the parameters of the normalized cut’s edge weight and see how they affect the eigenvector images and the segmentation result.
7. Compute image gradients on the first normalized cut’s eigenvectors. Combine these gradient images to detect image contours of objects.
8. Implement a linear search over the threshold value for the de-noised image in Chan-Vese segmentation. For each threshold, store the energy $E(\Gamma)$ and pick the segmentation with the lowest value.

This chapter gives a brief overview of how to use the popular computer vision library OpenCV through the Python interface. OpenCV is a C++ library for real-time computer vision initially developed by Intel and now maintained by Willow Garage. OpenCV is open source and released under a BSD license, meaning it is free for both academic and commercial use. As of version 2.0, Python support has been greatly improved. We will go through some basic examples and look deeper into tracking and video.

10.1 The OpenCV Python Interface

OpenCV is a C++ library with modules that cover many areas of computer vision. Besides C++ (and C), there is growing support for Python as a simpler scripting language through a Python interface on top of the C++ code base. The Python interface is still under development—not all parts of OpenCV are exposed and many functions are undocumented. This is likely to change, as there is an active community behind this interface. The Python interface is documented at <http://opencv.willowgarage.com/documentation/python/index.html>. See Appendix A for installation instructions.

The current OpenCV version (2.3.1) actually comes with two Python interfaces. The old `cv` module uses internal OpenCV datatypes and can be a little tricky to use from NumPy. The new `cv2` module uses NumPy arrays and is much more intuitive to use.¹ The module is available as

```
import cv2
```

and the old module can be accessed as

```
import cv2.cv
```

¹ The names and location of these two modules are likely to change over time. Check the online documentation for changes.

We will focus on the `cv2` module in this chapter. Look out for future name changes, as well as changes in function names and definitions in future versions. OpenCV and the Python interface is under rapid development.

10.2 OpenCV Basics

OpenCV comes with functions for reading and writing images, as well as matrix operations and math libraries. For the details on OpenCV, there is an excellent book [3] (C++ only). Let's look at some of the basic components and how to use them.

Reading and Writing Images

This short example will load an image, print the size, and convert and save the image in .png format:

```
import cv2

# read image
im = cv2.imread('empire.jpg')
h,w = im.shape[:2]
print h,w

# save image
cv2.imwrite('result.png',im)
```

The function `imread()` returns the image as a standard NumPy array and can handle a wide range of image formats. You can use this function as an alternative to the PIL image reading if you like. The function `imwrite()` automatically takes care of any conversion based on the file ending.

Color Spaces

In OpenCV images are not stored using the conventional RGB color channels; they are stored in BGR order (the reverse order). When reading an image the default is BGR; however, there are several conversions available. Color space conversions are done using the function `cvtColor()`. For example, converting to grayscale is done like this:

```
im = cv2.imread('empire.jpg')
# create a grayscale version
gray = cv2.cvtColor(im,cv2.COLOR_BGR2GRAY)
```

After the source image, there is an OpenCV color conversion code. Some of the most useful conversion codes are:

- `cv2.COLOR_BGR2GRAY`
- `cv2.COLOR_BGR2RGB`
- `cv2.COLOR_GRAY2BGR`

In each of these, the number of color channels for resulting images will match the conversion code (single channel for gray and three channels for RGB and BGR). The last

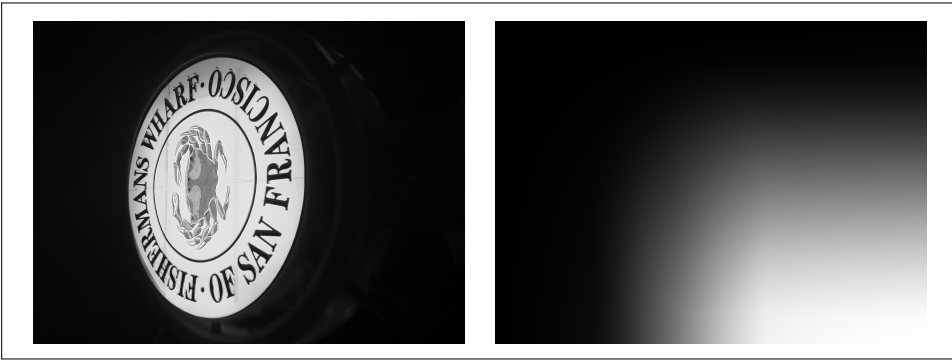


Figure 10-1. Example of computing an integral image using OpenCV's `integral()` function.

version converts grayscale images to BGR and is useful if you want to plot or overlay colored objects on the images. We will use this in the examples.

Displaying Images and Results

Let's look at some examples of using OpenCV for image processing and how to show results with OpenCV plotting and window management.

The first example reads an image from file and creates an integral image representation:

```
import cv2

# read image
im = cv2.imread('fisherman.jpg')
gray = cv2.cvtColor(im, cv2.COLOR_BGR2GRAY)

# compute integral image
intim = cv2.integral(gray)

# normalize and save
intim = (255.0 * intim) / intim.max()
cv2.imwrite('result.jpg', intim)
```

After reading the image and converting to grayscale, the function `integral()` creates an image where the value at each pixel is the sum of the intensities above and to the left. This is a very useful trick for quickly evaluating features. Integral images are used in OpenCV's `CascadeClassifier`, which is based on a framework introduced by Viola and Jones [39]. Before saving the resulting image, we normalize the values to 0 . . . 255 by dividing with the largest value. Figure 10-1 shows the result for an example image.

The second example applies flood filling starting from a seed pixel:

```
import cv2

# read image
filename = 'fisherman.jpg'
im = cv2.imread(filename)
h, w = im.shape[:2]
```

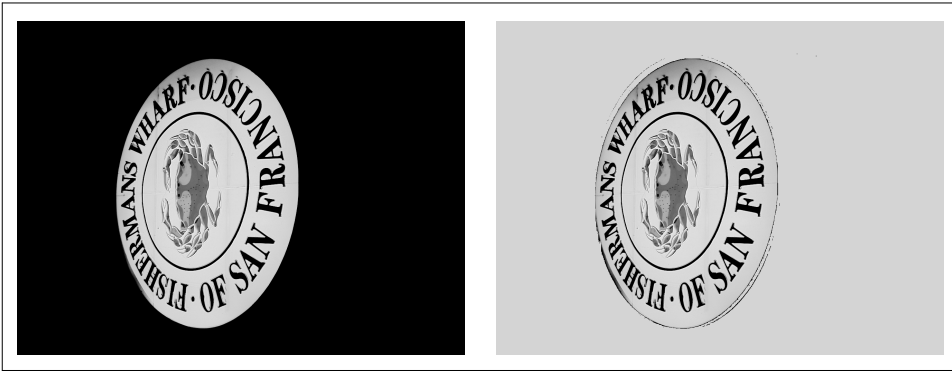


Figure 10-2. Flood fill of a color image. The highlighted area in the right panel marks all pixels filled using a single seed in the upper-left corner.

```
# flood fill example
diff = (6,6,6)
mask = zeros((h+2,w+2),uint8)
cv2.floodFill(im,mask,(10,10), (255,255,0),diff,diff)

# show the result in an OpenCV window
cv2.imshow('flood fill',im)
cv2.waitKey()

# save the result
cv2.imwrite('result.jpg',im)
```

This example applies flood fill to the image and shows the result in an OpenCV window. The function `waitKey()` pauses until a key is pressed and the window is automatically closed. Here the function `floodFill()` takes the image (grayscale or color), a mask with non-zero pixels indicating areas not to be filled, a seed pixel, and the new color value to replace the flooded pixels together with lower and upper difference thresholds to accept new pixels. The flood fill starts at the seed pixel and keeps expanding as long as new pixels can be added within the difference thresholds. The difference thresholds are given as tuples (R,G,B). The result looks like Figure 10-2.

As a third and final example, we look at extracting SURF features, a faster version of SIFT introduced by [1]. Here we also show how to use some basic OpenCV plotting commands:

```
import cv2

# read image
im = cv2.imread('empire.jpg')

# downsample
im_lowres = cv2.pyrDown(im)

# convert to grayscale
gray = cv2.cvtColor(im_lowres,cv2.COLOR_RGB2GRAY)
```



Figure 10-3. Sample SURF features extracted and plotted using OpenCV.

```
# detect feature points
s = cv2.SURF()
mask = uint8(ones(gray.shape))
keypoints = s.detect(gray,mask)

# show image and points
vis = cv2.cvtColor(gray,cv2.COLOR_GRAY2BGR)

for k in keypoints[::10]:
    cv2.circle(vis,(int(k.pt[0]),int(k.pt[1])),2,(0,255,0),-1)
    cv2.circle(vis,(int(k.pt[0]),int(k.pt[1])),int(k.size),(0,255,0),2)

cv2.imshow('local descriptors',vis)
cv2.waitKey()
```

After reading the image, it is downsampled using the function `pyrDown()`, which, if no new size is given, creates a new image half the size of the original. Then the image is converted to grayscale and passed to a SURF keypoint detection object. The *mask* determines what areas to apply the keypoint detector. To plot, we convert the grayscale image to a color image and use the green channel for plotting the keypoints. We loop over every tenth keypoint and plot a circle at the center and one circle showing the scale (size) of the keypoint. The plotting function `circle()` takes an image, a tuple with image coordinates (integer only), a radius, a tuple with plot color, and finally the line thickness (`-1` gives a solid circle). Figure 10-3 shows the result.

10.3 Processing Video

Video with pure Python is hard. There are speed, codecs, cameras, operating systems, and file formats to consider. There is currently no video library for Python. OpenCV

with its Python interface is the only good option. In this section, we'll look at some basic examples using video.

Video Input

Reading video from a camera is very well supported in OpenCV. A basic complete example that captures frames and shows them in an OpenCV window looks like this:

```
import cv2

# setup video capture
cap = cv2.VideoCapture(0)

while True:
    ret,im = cap.read()
    cv2.imshow('video test',im)
    key = cv2.waitKey(10)
    if key == 27:
        break
    if key == ord(' '):
        cv2.imwrite('vid_result.jpg',im)
```

The capture object `VideoCapture` captures video from cameras or files. Here we pass an integer at initialization. This is the id of the video device; with a single camera connected this is 0. The method `read()` decodes and returns the next video frame. The first value is a success flag and the second the actual image array. The `waitKey()` function waits for a key to be pressed and quits the application if the 'Esc' key (Ascii number 27) is pressed, or saves the frame if the 'space' key is pressed.

Let's extend this example with some simple processing by taking the camera input and showing a blurred (color) version of the input in an OpenCV window. This is only a slight modification to the base example above:

```
import cv2

# setup video capture
cap = cv2.VideoCapture(0)

# get frame, apply Gaussian smoothing, show result
while True:
    ret,im = cap.read()
    blur = cv2.GaussianBlur(im,(0,0),5)
    cv2.imshow('camera blur',blur)
    if cv2.waitKey(10) == 27:
        break
```

Each frame is passed to the function `GaussianBlur()`, which applies a Gaussian filter to the image. In this case, we are passing a color image so each color channel is blurred separately. The function takes a tuple for filter size and the standard deviation for the Gaussian function (in this case 5). If the filter size is set to zero, it will automatically be determined from the standard deviation. The result looks like Figure 10-4.

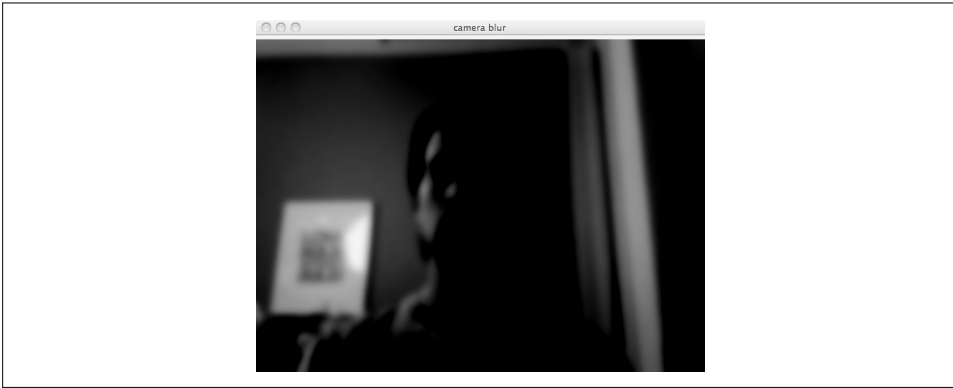


Figure 10-4. Screenshot of a blurred video of the author as he's writing this chapter.

Reading video from files works the same way but with the call to `VideoCapture()` taking the video filename as input:

```
capture = cv2.VideoCapture('filename')
```

Reading Video to NumPy Arrays

Using OpenCV, it is possible to read video frames from a file and convert them to NumPy arrays. Here is an example of capturing video from a camera and storing the frames in a NumPy array:

```
import cv2

# setup video capture
cap = cv2.VideoCapture(0)

frames = []
# get frame, store in array
while True:
    ret, im = cap.read()
    cv2.imshow('video', im)
    frames.append(im)
    if cv2.waitKey(10) == 27:
        break
frames = array(frames)

# check the sizes
print im.shape
print frames.shape
```

Each frame array is added to the end of a list until the capturing is stopped. The resulting array will have size (number of frames, height, width, 3). The printout confirms this:

```
(480, 640, 3)
(40, 480, 640, 3)
```

In this case, there were 40 frames recorded. Arrays with video data like this are useful for video processing, such as in computing frame differences and tracking.

10.4 Tracking

Tracking is the process of following objects through a sequence of images or video.

Optical Flow

Optical flow (sometimes called *optic flow*) is the image motion of objects as the objects, scene, or camera move between two consecutive images. It is a 2D vector field of within-image translation. It is a classic and well-studied field in computer vision with many successful applications in, for example, video compression, motion estimation, object tracking, and image segmentation.

Optical flow relies on three major assumptions:

1. Brightness constancy: The pixel intensities of an object in an image do not change between consecutive images.
2. Temporal regularity: The between-frame time is short enough to consider the motion change between images using differentials (used to derive the central equation below).
3. Spatial consistency: Neighboring pixels have similar motion.

In many cases, these assumptions break down, but for small motions and short time steps between images it is a good model. Assuming that an object pixel $I(x, y, t)$ at time t has the same intensity at time $t + \delta t$ after motion $[\delta x, \delta y]$ means that $I(x, y, t) = I(x + \delta x, y + \delta y, t + \delta t)$. Differentiating this constraint gives the *optical flow equation*:

$$\nabla I^T \mathbf{v} = -I_t,$$

where $\mathbf{v} = [u, v]$ is the motion vector and I_t the time derivative. For individual points in the image, this equation is under-determined and cannot be solved (one equation with two unknowns in $\mathbf{v}$). By enforcing some spatial consistency, it is possible to obtain solutions, though. In the Lucas-Kanade algorithm below, we will see how that assumption is used.

OpenCV contains several optical flow implementations: `CalcOpticalFlowBM()`, which uses block matching; `CalcOpticalFlowHS()`, which uses [15] (both of these currently only in the old `cv` module); the pyramidal Lucas-Kanade algorithm [19] `calcOpticalFlowPyrLK()`; and finally, `calcOpticalFlowFarneback()` based on [10]. The last one is considered one of the best methods for obtaining dense flow fields. Let's look at an example of using this to find motion vectors in video (the Lucas-Kanade version is the subject of the next section).

Try running the following script:

```
import cv2

def draw_flow(im, flow, step=16):
    """ Plot optical flow at sample points
        spaced step pixels apart. """

    h, w = im.shape[:2]
```

```

y,x = mgrid[step/2:h:step,step/2:w:step].reshape(2,-1)
fx,fy = flow[y,x].T

# create line endpoints
lines = vstack([x,y,x+fx,y+fy]).T.reshape(-1,2,2)
lines = int32(lines)

# create image and draw
vis = cv2.cvtColor(im,cv2.COLOR_GRAY2BGR)
for (x1,y1),(x2,y2) in lines:
    cv2.line(vis,(x1,y1),(x2,y2),(0,255,0),1)
    cv2.circle(vis,(x1,y1),1,(0,255,0), -1)
return vis

# setup video capture
cap = cv2.VideoCapture(0)

ret,im = cap.read()
prev_gray = cv2.cvtColor(im,cv2.COLOR_BGR2GRAY)

while True:
    # get grayscale image
    ret,im = cap.read()
    gray = cv2.cvtColor(im,cv2.COLOR_BGR2GRAY)

    # compute flow
    flow = cv2.calcOpticalFlowFarneback(prev_gray,gray,None,0.5,3,15,3,5,1.2,0)
    prev_gray = gray

    # plot the flow vectors
    cv2.imshow('Optical flow',draw_flow(gray,flow))
    if cv2.waitKey(10) == 27:
        break

```

This example will capture images from a webcam and call the optical flow estimation on every consecutive pair of images. The motion flow vectors are stored in the two-channel image *flow* returned by `calcOpticalFlowFarneback()`. Besides the previous frame and the current frame, this function takes a sequence of parameters. Look them up in the documentation if you are interested. The helper function `draw_flow()` plots the motion vectors at evenly spaced points in the image. It uses the OpenCV drawing functions `line()` and `circle()`, and the variable *step* controls the spacing of the flow samples. The result can look like the screenshots in Figure 10-5. Here the positions of the flow samples are shown as a grid of circles and the flow vectors with lines show how each sample point moves.

The Lucas-Kanade Algorithm

The most basic form of tracking is to follow interest points such as corners. A popular algorithm for this is the *Lucas-Kanade tracking algorithm*, which uses a sparse optical flow algorithm.

Lucas-Kanade tracking can be applied to any type of features, but usually makes use of corner points similar to the Harris corner points in Section 2.1. The function

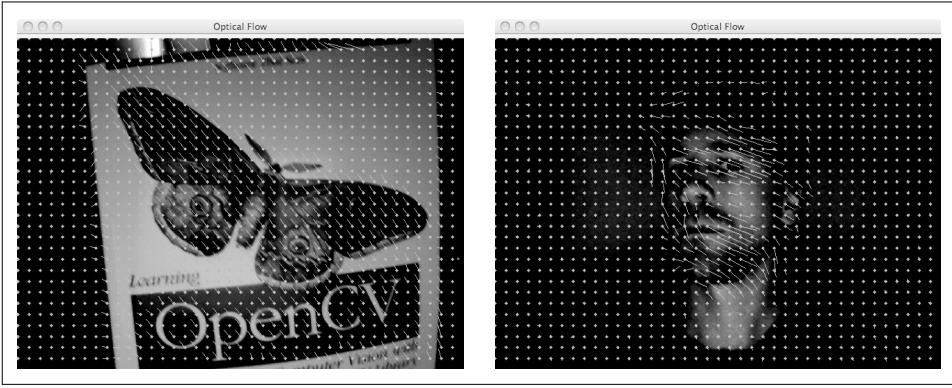


Figure 10-5. Optical flow vectors (sampled at every 16th pixel) shown on video of a translating book and a turning head.

`goodFeaturesToTrack()` detects corners according to an algorithm by Shi and Tomasi [33], where corners are points with two large eigenvalues of the structure tensor (Harris matrix) equation (2.2) and where the smaller eigenvalue is above a threshold.

The optical flow equation is under-determined (meaning that there are too many unknowns per equation) if considered on a per-pixel basis. Using the assumption that neighboring pixels have the same motion, it is possible to stack many of these equations into one system of equations like this

$$\begin{bmatrix} \nabla I^T(\mathbf{x}_1) \\ \nabla I^T(\mathbf{x}_2) \\ \vdots \\ \nabla I^T(\mathbf{x}_n) \end{bmatrix} \mathbf{v} = \begin{bmatrix} I_x(\mathbf{x}_1) & I_y(\mathbf{x}_1) \\ I_x(\mathbf{x}_2) & I_y(\mathbf{x}_2) \\ \vdots & \vdots \\ I_x(\mathbf{x}_n) & I_y(\mathbf{x}_n) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} I_t(\mathbf{x}_1) \\ I_t(\mathbf{x}_2) \\ \vdots \\ I_t(\mathbf{x}_n) \end{bmatrix}$$

for some neighborhood of n pixels. This has the advantage that the system now has more equations than unknowns and can be solved with least square methods. Typically, the contribution from the surrounding pixels is weighted so that pixels farther away have less influence. A Gaussian weighting is the most common choice. This turns the matrix above into the structure tensor in equation (2.2), and we have the relation

$$\bar{\mathbf{M}}_I \mathbf{v} = - \begin{bmatrix} I_t(\mathbf{x}_1) \\ I_t(\mathbf{x}_2) \\ \vdots \\ I_t(\mathbf{x}_n) \end{bmatrix} \quad \text{or simpler } \mathbf{A} \mathbf{v} = \mathbf{b}.$$

This over-determined equation system can be solved in a least square sense and the motion vector is given by

$$\mathbf{v} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{b}.$$

This is solvable only when $\mathbf{A}^T \mathbf{A}$ is invertible, which it is by construction if applied at Harris corner points or the “good features to track” of Shi-Tomasi. This is how the motion vectors are computed in the Lucas-Kanade tracking algorithms.

Standard Lucas-Kanade tracking works for small displacements. To handle larger displacements, a hierarchical approach is used. In this case, the optical flow is computed at coarse-to-fine versions of the image. This is what the OpenCV function `calcOpticalFlowPyrLK()` does.

The Lucas-Kanade functions are included in OpenCV. Let's look at how to use those to build a Python tracker class. Create a file *lktrack.py* and add the following class and constructor:

```
import cv2

# some constants and default parameters
lk_params = dict(winSize=(15,15),maxLevel=2,
                  criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT,10,0.03))

subpix_params = dict(zeroZone=(-1,-1),winSize=(10,10),
                     criteria = (cv2.TERM_CRITERIA_COUNT | cv2.TERM_CRITERIA_EPS,20,0.03))

feature_params = dict(maxCorners=500,qualityLevel=0.01,minDistance=10)

class LKTracker(object):
    """ Class for Lucas-Kanade tracking with
        pyramidal optical flow. """

    def __init__(self,imnames):
        """ Initialize with a list of image names. """

        self.imnames = imnames
        self.features = []
        self.tracks = []
        self.current_frame = 0
```

The tracker object is initialized with a list of filenames. The variables *features* and *tracks* will hold the corner points and their tracked positions. We also use a variable to keep track of the current frame. We define three dictionaries with parameters for the feature extraction, the tracking, and the subpixel feature point refinement.

Now, to start detecting points, we need to load the actual image, create a grayscale version, and extract the “good features to track” points. The OpenCV function doing the main work is `goodFeaturesToTrack()`. Add this `detect_points()` method to the class:

```
def detect_points(self):
    """ Detect 'good features to track' (corners) in the current frame
        using sub-pixel accuracy. """

    # load the image and create grayscale
    self.image = cv2.imread(self.imnames[self.current_frame])
    self.gray = cv2.cvtColor(self.image,cv2.COLOR_BGR2GRAY)

    # search for good points
    features = cv2.goodFeaturesToTrack(self.gray, **feature_params)
```

```

# refine the corner locations
cv2.cornerSubPix(self.gray, features, **subpix_params)

self.features = features
self.tracks = [[p] for p in features.reshape((-1,2))]

self.prev_gray = self.gray

```

The point locations are refined using `cornerSubPix()` and stored in the member variables *features* and *tracks*. Note that running this function clears the track history.

Now that we can detect the points, we also need to track them. First, we need to get the next frame, apply the OpenCV function `calcOpticalFlowPyrLK()` that finds out where the points moved, and then remove and clean the lists of tracked points. The method `track_points()` below does this:

```

def track_points(self):
    """ Track the detected features. """

    if self.features != []:
        self.step() # move to the next frame

        # load the image and create grayscale
        self.image = cv2.imread(self.imnames[self.current_frame])
        self.gray = cv2.cvtColor(self.image, cv2.COLOR_BGR2GRAY)

        # reshape to fit input format
        tmp = float32(self.features).reshape(-1, 1, 2)

        # calculate optical flow
        features, status, track_error = cv2.calcOpticalFlowPyrLK(self.prev_gray,
                                                                self.gray, tmp, None, **lk_params)

        # remove points lost
        self.features = [p for (st, p) in zip(status, features) if st]

        # clean tracks from lost points
        features = array(features).reshape((-1, 2))
        for i, f in enumerate(features):
            self.tracks[i].append(f)
        ndx = [i for (i, st) in enumerate(status) if not st]
        ndx.reverse() # remove from back
        for i in ndx:
            self.tracks.pop(i)

        self.prev_gray = self.gray

```

This makes use of a simple helper method `step()` that moves to the next available frame:

```

def step(self, framenbr=None):
    """ Step to another frame. If no argument is
        given, step to the next frame. """

    if framenbr is None:
        self.current_frame = (self.current_frame + 1) % len(self.imnames)
    else:
        self.current_frame = framenbr % len(self.imnames)

```

This method jumps to a given frame or just to the next if no argument is given.

Finally, we also want to be able to draw the result using OpenCV windows and drawing functions. Add this `draw()` method to the `LKTracker` class:

```
def draw(self):
    """ Draw the current image with points using
        OpenCV's own drawing functions.
        Press ant key to close window."""

    # draw points as green circles
    for point in self.features:
        cv2.circle(self.image,(int(point[0][0]),int(point[0][1])),3,(0,255,0),-1)

    cv2.imshow('LKtrack',self.image)
    cv2.waitKey()
```

Now we have a complete self-contained tracking system using OpenCV functions.

Using the tracker

Let's tie it all together by using this tracker class on a real tracking scenario. The following script will initialize a tracker object, detect and track points through the sequence, and draw the result:

```
import lktrack

imnames = ['bt.003.pgm', 'bt.002.pgm', 'bt.001.pgm', 'bt.000.pgm']

# create tracker object
lkt = lktrack.LKTracker(imnames)

# detect in first frame, track in the remaining
lkt.detect_points()
lkt.draw()
for i in range(len(imnames)-1):
    lkt.track_points()
    lkt.draw()
```

The drawing is one frame at a time and shows the points currently tracked. Pressing any key will move to the next image in the sequence. The resulting figure windows for the first four images of the Oxford corridor sequence (one of the Oxford multi-view datasets available at <http://www.robots.ox.ac.uk/~vgg/data/data-mview.html>) looks like Figure 10-6.

Using generators

Add the following method to the `LKTracker` class:

```
def track(self):
    """ Generator for stepping through a sequence."""

    for i in range(len(self.imnames)):
        if self.features == []:
            self.detect_points()
        else:
            self.track_points()
```



Figure 10-6. Tracking using the Lucas-Kanade algorithm through the LKTrack class.

```
# create a copy in RGB
f = array(self.features).reshape(-1,2)
im = cv2.cvtColor(self.image,cv2.COLOR_BGR2RGB)
yield im,f
```

This creates a generator that makes it easy to step through a sequence and get tracks and the images as RGB arrays so that it is easy to plot the result. To use it on the classic Oxford “dinosaur” sequence (from the same multi-view dataset page as the corridor above) and plot the points and their tracks, the code looks like this:

```
import lktrack

imnames = ['viff.000.ppm', 'viff.001.ppm',
           'viff.002.ppm', 'viff.003.ppm', 'viff.004.ppm']

# track using the LKTracker generator
lkt = lktrack.LKTracker(imnames)
for im,ft in lkt.track():
    print 'tracking %d features' % len(ft)
```

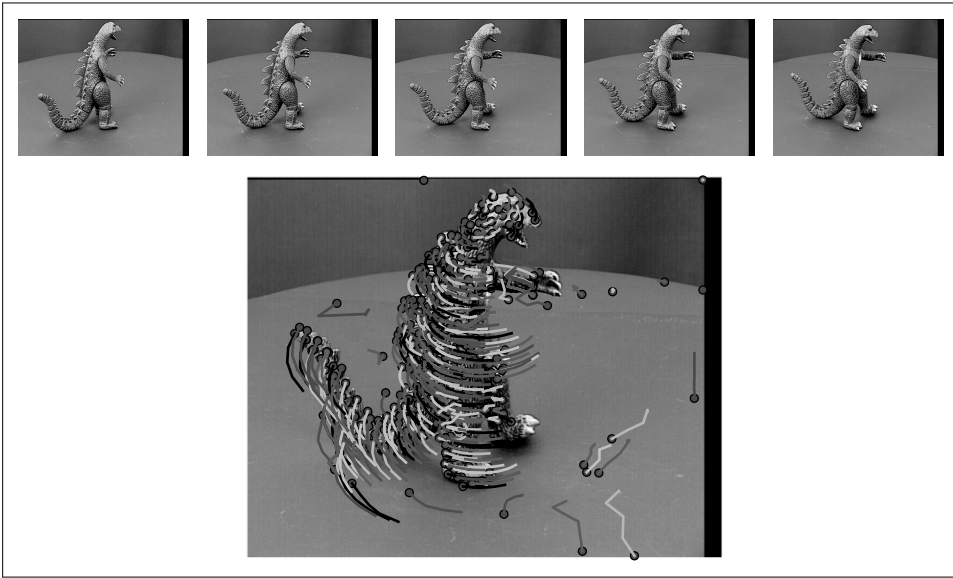


Figure 10-7. An example of using Lucas-Kanade tracking on a turntable sequence and plotting the tracks of points.

```
# plot the tracks
figure()
imshow(im)
for p in ft:
    plot(p[0],p[1],'bo')
for t in lkt.tracks:
    plot([p[0] for p in t],[p[1] for p in t])
axis('off')
show()
```

This generator makes it really easy to use the tracker class and completely hides the OpenCV functions from the user. The example generates a plot like the one shown in Figure 10-7 and the bottom right of Figure 10-6.

10.5 More Examples

With OpenCV comes a number of useful examples of how to use the Python interface. These are in the sub-directory `samples/python2/` and are a good way to get familiar with OpenCV. Here are a few selected examples to illustrate some other capabilities of OpenCV.

Inpainting

The reconstruction of lost or deteriorated parts of images is called *inpainting*. This covers both algorithms to recover lost or corrupted parts of image data for restoration purposes as well as removing red-eyes or objects in photo-editing applications.



Figure 10-8. An example of inpainting with OpenCV. The left image shows areas marked by a user as “corrupt.” The right image shows the result after inpainting.

Typically, a region of the image is marked as “corrupt” and needs to be filled using the data from the rest of the image.

Try the following command:

```
$ python inpaint.py empire.jpg
```

This will open an interactive window where you can draw regions to be inpainted. The results are shown in a separate window. An example is shown in Figure 10-8.

Segmentation with the Watershed Transform

Watershed is an image processing technique that can be used for segmentation (see Figure 10-9). An image is treated as a topological landscape that is “flooded” from a number of seed regions. Usually, a gradient magnitude image is used since this has ridges at strong edges and will make the segmentation stop at image edges.

The implementation in OpenCV uses an algorithm by Meyer [22]. Try it using the following command:

```
$ python watershed.py empire.jpg
```

This will open an interactive window where you can draw the seed regions you want the algorithm to use as input. The results are shown in a second window with colors representing regions overlaid on a grayscale version of the input image.

Line Detection with a Hough Transform

The *Hough transform* (http://en.wikipedia.org/wiki/Hough_transform) is a method for finding shapes in images. It works by using a voting procedure in the parameter space of the shapes. The most common use is to find line structures in images. In that case,



Figure 10-9. An example of segmenting an image using a watershed transform. The left image is the input image with seed regions drawn. The right image shows the resulting segmentation with colors overlaid on the image.



Figure 10-10. An example of detecting lines using a Hough transform. The left image is the source in grayscale. The right image shows an edge map with detected lines.

edges and line segments can be grouped together by them voting for the same line parameters in the 2D parameter space of lines.

The OpenCV sample detects lines using this approach.² Try the following command:

```
$ python houghlines.py empire.jpg
```

This gives two windows like the ones shown in Figure 10-10. One window shows the source image in grayscale, and the other shows the edge map used together with lines

² This sample is currently in the /samples/python folder.

detected as those with most votes in parameter space. Note that the lines are always infinite; if you want to find the endpoints of line segments in the image, you can use the edge map to try to find them.

Exercises

1. Use optical flow to build a simple gesture recognition system. For example, you could sample the flow as in the plotting function and use these sample vectors as input.
2. There are two warp functions available in OpenCV, `cv2.warpAffine()` and `cv2.warpPerspective()`. Try to use them on some of the examples from Chapter 3.
3. Use the flood fill function to do background subtraction on the Oxford “dinosaur” images used in Figure 10-7. Create new images with the dinosaur placed on a different color background or on a different image.
4. OpenCV has a function `cv2.findChessboardCorners()`, which automatically finds the corners of a chessboard pattern. Use this function to get correspondences for calibrating a camera with the function `cv2.calibrateCamera()`.
5. If you have two cameras, mount them in a stereo rig setting and capture stereo image pairs using `cv2.VideoCapture()` with different video device ids. Try 0 and 1 for starters. Compute depth maps for some varying scenes.
6. Use Hu moments with `cv2.HuMoments()` as features for the Sudoku OCR classification problem in Section 8.4 and check the performance.
7. OpenCV has an implementation of the Grab Cut segmentation algorithm. Use the function `cv2.grabCut()` on the Microsoft Research Grab Cut dataset (see Section 9.1). Hopefully you will get better results than the low-resolution segmentation in our examples.
8. Modify the Lucas-Kanade tracker class to take a video file as input and write a script that tracks points between frames and detects new points every k frames.

Installing Packages

Here are short installation instructions for the packages used in the book. They are written based on the latest versions as of the writing of this book. Things change (URLs change!), so if the instructions become outdated, check the individual project websites for help.

In addition to the specific instructions, an option that often works on most platforms is Python's `easy_install`. If you run into problems with the installation instructions given here, `easy_install` is worth a try. Find out more on the package website, http://packages.python.org/distribute/easy_install.html.

A.1 NumPy and SciPy

Installing NumPy and SciPy is a little different depending on your operating system. Follow the applicable instructions below. The current versions are 2.0 (NumPy) and 0.11 (SciPy) on most platforms. A package that currently works on all major platforms is the Enthought EPD Free bundle, a free light version of the commercial Enthought distribution, available at http://enthought.com/products/epd_free.php.

Windows

The easiest way to install NumPy and SciPy is to download and install the binary distributions from <http://www.scipy.org/Download>.

Mac OS X

Later versions of Mac OS X (10.7.0 [Lion] and up) come with NumPy pre-installed.

An easy way to install NumPy and SciPy for Mac OS X is with the “superpack” from <https://github.com/fonnesbeck/ScipySuperpack>. This also gives you Matplotlib.

Another alternative is to use the package system MacPorts (<http://www.macports.org/>). This also works for Matplotlib instead of the instructions below.

If none of those work, the project web page has other alternatives listed (<http://scipy.org/>).

Linux

Installation requires that you have administrator rights on your computer. On some distributions NumPy comes pre-installed, on others not. Both NumPy and SciPy are most easily installed with the built-in package handler (for example Synaptic on Ubuntu). You can also use the package handler for Matplotlib instead of the instructions below.

A.2 Matplotlib

Here are instructions for installing Matplotlib in case your NumPy/SciPy installation did not also install Matplotlib. Matplotlib is freely available at <http://matplotlib.sourceforge.net/>. Click the “download” link and download the installer for the latest version for your system and Python version. Currently the latest version is 1.1.0.

Alternatively, just download the source and unpack. Run

```
$ python setup.py install
```

from the command line and everything should work. General tips on installing for different systems can be found at <http://matplotlib.sourceforge.net/users/installing.html>, but the process above should work for most platforms and Python versions.

A.3 PIL

PIL, the Python Imaging Library, is available at <http://www.pythonware.com/products/pil/>. The latest free version is 1.1.7. Download the source kit and unpack the folder. In the downloaded folder, run

```
$ python setup.py install
```

from the command line.

You need to have JPEG (libjpeg) and PNG (zlib) supported if you want to save images using PIL. See the README file or the PIL website if you encounter any problems.

A.4 LibSVM

The current release is version 3.1 (released April 2011). Download the zip file from the LibSVM website (<http://www.csie.ntu.edu.tw/~cjlin/libsvm/>). Unzip the file (a directory “libsvm-3.1” will be created). In a terminal window, go to this directory and type “make”:

```
$ cd libsvm-3.0
$ make
```

Then go to the “python” directory and do the same:

```
$ cd python/
$ make
```

This should be all you need to do. To test your installation, start Python from the command line and try:

```
import svm
```

The authors wrote a practical guide for using LivSVM [7]. This is a good starting point.

A.5 OpenCV

Installing OpenCV is a bit different, depending on your operating system. Follow the applicable instructions below.

To check your installation, start Python and try the cookbook examples at <http://opencv.willowgarage.com/documentation/python/cookbook.html>. The online OpenCV Python reference guide gives more examples and details on how to use OpenCV with Python at <http://opencv.willowgarage.com/documentation/python/index.html>.

Windows and Unix

There are installers for Windows and Unix available at the SourceForge repository, <http://sourceforge.net/projects/opencvlibrary/>.

Mac OS X

Mac OS X support has been lacking but is on the rise. There are several ways to install from source as described on the OpenCV wiki, <http://opencv.willowgarage.com/wiki/InstallGuide>. MacPorts is one option that works well if you are using Python, NumPy, SciPy, or Matplotlib, also from MacPorts. Building OpenCV from source can be done like this:

```
$ svn co https://code.ros.org/svn/opencv/trunk/opencv
$ cd opencv/
$ sudo cmake -G "Unix Makefiles" .
$ sudo make -j8
$ sudo make install
```

If you have all the dependencies in place, everything should build and install properly. If you get an error like

```
import cv2
Traceback (most recent call last):
  File "", line 1, in
ImportError: No module named cv2
```

then you need to add the directory containing cv2.so to PYTHONPATH. For example:

```
$ export PYTHONPATH=/usr/local/lib/python2.7/site-packages/
```

Linux

Linux users could try the package installer for the distribution (the package is usually called “opencv”) or install from source as described in the Mac OS X section.

A.6 VLFeat

To install VLFeat, download and unpack the latest binary package from <http://vlfeat.org/download.html> (currently the latest version is 0.9.14). Add the paths to your environment or copy the binaries to a directory in your path. The binaries are in the bin/ directory, just pick the sub-directory for your platform.

The use of the VLFeat command line binaries is described in the src/ sub-directory. Alternatively, you can find the documentation online at <http://vlfeat.org/man/man.html>.

A.7 PyGame

PyGame can be downloaded from <http://www.pygame.org/download.shtml>. The latest version is 1.9.1. The easiest way is to get the binary install package for your system and Python version.

Alternatively, you can download the source, and in the downloaded folder run

```
$ python setup.py install
```

from the command line.

A.8 PyOpenGL

Installing PyOpenGL is easiest done by downloading the package from <http://pypi.python.org/pypi/PyOpenGL> as suggested on the PyOpenGL web page, <http://pyopengl.sourceforge.net/>. Get the latest version, currently 3.0.1.

In the downloaded folder, do the usual

```
$ python setup.py install
```

from the command line. If you get stuck or need information on dependencies, etc., more documentation can be found at <http://pyopengl.sourceforge.net/documentation/installation.html>. Some good demo scripts for getting started are available at <http://pypi.python.org/pypi/PyOpenGL-Demo>.

A.9 Pydot

Begin by installing the dependencies, GraphViz and Pyparsing. Go to <http://www.graphviz.org/> and download the latest GraphViz binary for your platform. The install files should install GraphViz automatically.

Then go to the Pyparsing project page <http://pyparsing.wikispaces.com/>. The download page is at <http://sourceforge.net/projects/pyparsing/>. Get the latest version (currently 1.5.5) and unzip the file to a directory. Type

```
$ python setup.py install
```

from the command line.

Finally, go to the project page <http://code.google.com/p/pydot/> and click “download”. From the download page, download the latest version (currently 1.0.4). Unzip and again type

```
$ python setup.py install
```

from the command line. Now you should be able to import pydot in your Python sessions.

A.10 Python-graph

Python-graph is a Python module for working with graphs and contains lots of useful algorithms like traversals, shortest path, pagerank, and maximum flow. The latest version is 1.8.1 and can be found on the project website <http://code.google.com/p/python-graph/>. If you have `easy_install` on your system, the simplest way to get python-graph is:

```
$ easy_install python-graph-core
```

Alternatively, download the source code from <http://code.google.com/p/python-graph/downloads/list> and run:

```
$ python setup.py install
```

To write and visualize the graphs (using the DOT language) you need python-graph-dot, which comes with the download or through `easy_install`:

```
$ easy_install python-graph-dot
```

Python-graph-dot depends on pydot; see above. The documentation (in html) is in the “docs/” folder.

A.11 Simplejson

Simplejson is the independently maintained version of the JSON module that comes with later versions of Python (2.6 or later). The syntax is the same for both modules, but simplejson is more optimized and will give better performance.

To install, go to the project page <https://github.com/simplejson/simplejson> and click the Download button. Then select the latest version from the “Download Packages” section (currently this is 2.1.3). Unzip the folder and type

```
$ python setup.py install
```

from the command line. This should be all you need.

A.12 PySQLite

PySQLite is an SQLite binding for Python. SQLite is a lightweight, disk-based database that can be queried with SQL and is easy to install and use. The latest version is 2.6.3; see the project website, <http://code.google.com/p/pysqlite/>, for more details.

To install, download from <http://code.google.com/p/pysqlite/downloads/list> and unzip to a folder. Run

```
$ python setup.py install
```

from the command line.

A.13 CherryPy

CherryPy (<http://www.cherrypy.org/>) is a fast, stable, and lightweight web server built on Python using an object-oriented model. CherryPy is easy to install; just download the latest version from <http://www.cherrypy.org/wiki/CherryPyInstall>. The latest stable release is 3.2.0. Unpack and run

```
$ python setup.py install
```

from the command line. After installing, look at the tiny tutorial examples that come with CherryPy in the *cherrypy/tutorial/* folder. These examples show you how to pass GET/POST variables, inheritance of page properties, file upload and download, etc.

Image Datasets

B.1 Flickr

The immensely popular photo-sharing site Flickr (<http://flickr.com/>) is a gold mine for computer vision researchers and hobbyists. With hundreds of millions of images, many of them tagged by users, it is a great resource to get training data or for doing experiments on real data. Flickr has an API for interfacing with the service that makes it possible to upload, download, and annotate images (and much more). A full description of the API is available at <http://flickr.com/services/api/>, and there are kits for many programming languages, including Python.

Let's look at using a library called flickrpy, available freely at <http://code.google.com/p/flickrpy/>. Download the file *flickr.py*. You will need an API Key from Flickr to get this to work. Keys are free for non-commercial use and can be requested for commercial use. Just click the link "Apply for a new API Key" on the Flickr API page and follow the instructions. Once you have an API key, open *flickr.py* and replace the empty string on the line

```
API_KEY = ''
```

with your key. It should look something like this:

```
API_KEY = '123fbbb81441231123cgg5b123d92123'
```

Let's create a simple command line tool that downloads images tagged with a particular tag. Add the following code to a new file called *tagdownload.py*:

```
import flickr
import urllib, urlparse
import os
import sys

if len(sys.argv)>1:
    tag = sys.argv[1]
else:
    print 'no tag specified'
```

```
# downloading image data
f = flickr.photos_search(tags=tag)
urllist = [] #store a list of what was downloaded

# downloading images
for k in f:
    url = k.getURL(size='Medium', urlType='source')
    urllist.append(url)
    image = urllib.URLopener()
    image.retrieve(url, os.path.basename(urlparse.urlparse(url).path))
    print 'downloading:', url
```

If you also want to write the list of urls to a text file, add the following lines at the end:

```
# write the list of urls to file
fl = open('urllist.txt', 'w')
for url in urllist:
    fl.write(url+'\n')
fl.close()
```

From the command line, just type

```
$ python tagdownload.py goldengatebridge
```

and you will get the 100 latest images tagged with “goldengatebridge”. As you can see, we chose to take the “Medium” size. If you want thumbnails or full-size originals or something else, there are many sizes available; check the documentation on the Flickr website, <http://flickr.com/api/>.

Here we were just interested in downloading images; for API calls that require authentication the process is slightly more complicated. See the API documentation for more information on how to set up authenticated sessions.

B.2 Panoramio

A good source of geotagged images is Google’s photo-sharing service Panoramio (<http://www.panoramio.com/>). This web service has an API to access content programmatically. The API is described at <http://www.panoramio.com/api/>. You can get website widgets and access the data using JavaScript objects. To download images, the simplest way is to use a GET call. For example:

```
http://www.panoramio.com/map/get_panoramas.php?order=popularity&set=public&
from=0&to=20&minx=-180&miny=-90&maxx=180&maxy=90&size=medium
```

where *minx*, *miny*, *maxx*, *maxy* define the geographic area to select photos from (minimum longitude, latitude, maximum longitude and latitude, respectively). The response will be in JSON and look like this:

```
{"count": 3152, "photos":
[{"upload_date": "02 February 2006", "owner_name": "****", "photo_id": 9439,
"longitude": -151.75, "height": 375, "width": 500, "photo_title": "****",
"latitude": -16.5, "owner_url": "http://www.panoramio.com/user/1600", "owner_id": 1600,
```

```

"photo_file_url": "http://mw2.google.com/mw-panoramio/photos/medium/9439.jpg",
"photo_url": "http://www.panoramio.com/photo/9439"},
{"upload_date": "18 January 2011", "owner_name": "***", "photo_id": 46752123,
"longitude": 120.52718600000003, "height": 370, "width": 500, "photo_title": "***",
"latitude": 23.327833999999999, "owner_url": "http://www.panoramio.com/user/2780232",
"owner_id": 2780232,
"photo_file_url": "http://mw2.google.com/mw-panoramio/photos/medium/46752123.jpg",
"photo_url": "http://www.panoramio.com/photo/46752123"},
{"upload_date": "20 January 2011", "owner_name": "***", "photo_id": 46817885,
"longitude": -178.13709299999999, "height": 330, "width": 500, "photo_title": "***",
"latitude": -14.310613, "owner_url": "http://www.panoramio.com/user/919358",
"owner_id": 919358,
"photo_file_url": "http://mw2.google.com/mw-panoramio/photos/medium/46817885.jpg",
"photo_url": "http://www.panoramio.com/photo/46817885"},
:
:
], "has_more": true}

```

Using a JSON package, you can get the “photo_file_url” field of the result. See Section 2.3 for an example.

B.3 Oxford Visual Geometry Group

The Visual Geometry research group at Oxford University has many datasets available at <http://www.robots.ox.ac.uk/~vgg/data/>. We used some of the multi-view datasets in this book, for example the “Merton1”, “Model House”, “dinosaur”, and “corridor” sequences. The data is available for download (some with camera matrices and point tracks) at <http://www.robots.ox.ac.uk/~vgg/data/data-mview.html>.

B.4 University of Kentucky Recognition Benchmark Images

The UK Benchmark image set, also called the “ukbench” set, is a set with 2,550 groups of images. Each group has four images of an object or scene from varying viewpoints. This is a good set to test object recognition and image retrieval algorithms. The data set is available for download (the full set is around 1.5 GB) at <http://www.vis.uky.edu/~stewe/ukbench/>. It is described in detail in the paper [23].

In this book, we used a smaller subset using only the first 1,000 images.

B.5 Other

Prague Texture Segmentation Datagenerator and Benchmark

This set used in the segmentation chapter can generate many different types of texture segmentation images. Available at <http://mosaic.utia.cas.cz/index.php>.

MSR Cambridge Grab Cut Dataset

Originally used in the Grab Cut paper [27], this set provides segmentation images with user annotations. The data set and some papers are available from <http://research.microsoft.com/en-us/um/cambridge/projects/visionimagevideoediting/segmentation/grab-cut.htm>. The original images in the data set are from a data set that now is part of the Berkeley Segmentation Dataset, <http://www.eecs.berkeley.edu/Research/Projects/CS/vision/grouping/segbench/>.

Caltech 101

This is a classic dataset that contains pictures of objects from 101 different categories and can be used to test object recognition algorithms. The data set is available at http://www.vision.caltech.edu/Image_Datasets/Caltech101/.

Static Hand Posture Database

This dataset from Sebastien Marcel is available at <http://www.idiap.ch/resource/gestures/> together with a few other sets with hands and gestures.

Middlebury Stereo Datasets

These are datasets used to benchmark stereo algorithms. They are available for download at <http://vision.middlebury.edu/stereo/data/>. Every stereo pair comes with ground truth depth images to compare results against.